Qwen

From Vibe to Verification: Enforcing Doctrinal Integrity in Horror.Place via a Unified AI-Powered Contract Framework

The `contractCard`: A Foundational Schema for Doctrinal Correctness

The cornerstone of the proposed framework is the `contractCard`, a formal, machine-readable document designed to translate a designer's creative intent into a set of verifiable constraints . This object serves as the single source of truth for a specific game element, such as a scene or seed, and acts as the lingua franca between human designers, AI assistants, procedural generation tools, and automated governance systems . Its primary purpose is to enforce doctrinal correctness, ensuring that all generated content adheres to predefined invariants and metric bands, thereby maintaining a consistent and high-quality horror experience . The architecture is explicitly designed to favor forward compatibility and doctrinal purity over strict backward compatibility, mandating a gentle migration path for legacy systems to align with the new standard .

The `contractCard` is formally defined by a JSON Schema, which provides the technical blueprint for its structure and constraints . This schema, intended to reside at `coreschemascontractcardv1.json`, ensures that every instance of a `contractCard` is uniform and predictable across the entire development ecosystem . The schema mandates several core properties. Every card must have a `title` to provide a human-readable label for the asset, a `styleid` to anchor it to a specific aesthetic and behavioral contract, and a `schemaref` to point to the underlying schema definition for the type of entity being created (e.g., a seed or event) . Critically, the schema also includes a placeholder for `deadledgerref`, which is essential for future ledger anchoring and auditability . By mirroring the fields found in established signature documents, the `contractCard` can leverage existing validation logic, making any content derived from it inherently "CI-ready by construction" .

The most crucial components of the `contractCard` are the `invariants` and `metrics` objects, which encode the quantitative essence of the design goal . The `invariants` object contains a set of six key values: CIC (Clue-to-Incident Ratio), AOS (Ambiguity-to-

Oppression Score), RRM (Resource-Restriction Metric), LSG (Locus-Safety Gradient), DET (Dread Event Threshold), and SHCI (Spectral-Harmonic Consistency Index). Each of these is a number with a specified range (e.g., `DET` ranges from 0.0 to 10.0), representing a target value for that attribute . The `metrics` object, in contrast, defines the permissible performance envelopes or bands for five other critical gameplay metrics: UEC (Uncanny Engagement Count), EMD (Environmental Dread Multiplier), STCI (Sudden Threat Confidence Interval), CDL (Creative Dread Level), and ARR (Audience Retention Rate). These metrics are expressed as `[min, max]` arrays, defining the acceptable range of outcomes for the associated seed or scene . This distinction is fundamental to the workflow: the AI proposes a specific point within an envelope (e.g., `DET: 3.5` inside the band `STCI: [0.0, 0.2]`), allowing the designer to interactively tweak the target value while keeping the overall design within safe, playable bounds .

The following table details the structure of the `contractCard` as defined by its corresponding JSON Schema.

Property	Type	Description
`title`	String	A human-readable title for the contract card. Must be between 1 and 160 characters.
`styleid`	String	A reference to a style contract that governs the visual and auditory tone. Must be between 1 and 128 characters.
`schemaref`	String	A reference to the schema that defines the structure of the entity this card creates. Must match the pattern `^horror\\\\.place\\\\.schema\\\\.v1:[a-z0-9_.-]+$`.
`invariants`	Object	An object containing the target values for core horror mechanics.
`invariants.CIC`	Number	Clue-to-Incident Ratio. Must be a float between 0.0 and 1.0.
`invariants.AOS`	Number	Ambiguity-to-Oppression Score. Must be a float between 0.0 and 1.0.
`invariants.RRM`	Number	Resource-Restriction Metric. Must be a float between 0.0 and 1.0.
`invariants.LSG`	Number	Locus-Safety Gradient. Must be a float between 0.0 and 1.0.
`invariants.DET`	Number	Dread Event Threshold. Must be a float between 0.0 and 10.0.
`invariants.SHCI`	Number	Spectral-Harmonic Consistency Index. Must be a float between 0.0 and 1.0.
`metrics`	Object	An object containing the acceptable `[min, max]` bands for outcome metrics.
`metrics.UEC`	Array	Uncanny Engagement Count band. Must be an array of two floats, each between 0.0 and 1.0.
`metrics.EMD`	Array	Environmental Dread Multiplier band. Must be an array of two floats, each between 0.0 and 1.0.
`metrics.STCI`	Array	Sudden Threat Confidence Interval band. Must be an array of two floats, each between 0.0 and 1.0.
`metrics.CDL`	Array	Creative Dread Level band. Must be an array of two floats, each between 0.0 and 1.0.
`metrics.ARR`	Array	Audience Retention Rate band. Must be an array of two floats, each between 0.0 and 1.0.
`notes`	String	A field for designers to add qualitative rationale for the chosen parameters. Can be empty, up to 2000 characters.
`deadledgerref`	String	A placeholder reference to a ledger entry for provenance and auditing. Must match the pattern `^deadledger://[a-z0-9_./-]+$`.

This schema-based approach is not merely a technical detail; it is the foundation of the entire doctrine. By treating the `contractCard` as a UI-friendly projection of the same invariant/metric envelopes used in first-pass signatures, the system reuses proven JSON Schema patterns and determinism rules, such as requiring explicit values without defaults and rejecting cards that would violate policy caps . This unification means that a `contractCard` proposed in the editor could, in principle, be wrapped into a full `signature.json` file for CI submission, ensuring consistency across the toolchain . The schema itself can be modularly composed using standards like `$ref` and `allOf`, allowing for the creation of more complex contract types (like regional envelopes) by extending the base `contractCard` schema, further promoting a single-source-of-truth philosophy [2] [72].

Architectural Implementation: Integrating the AI Copilot in the Unreal Editor

The implementation of the unified contract framework hinges on transforming the Unreal Editor into a sophisticated frontend for the Horror.Place doctrine, where the AI Copilot operates as a schema-aware co-designer rather than a simple content generator . This involves creating a tightly integrated system where every action taken by the AI is grounded in the `contractCard` and validated against the project's schemas and policies in real time . The interaction model is designed to shift the designer's workflow from abstract, "vibe"-based prompts to a structured, constraint-driven process, making the principles of the horror genre tangible and measurable . The initial phase of implementation focuses on building the core UI and validator, followed by progressively adding features like ledger queries and continuous drift detection .

The core interaction loop begins when a designer provides a natural language prompt, such as "Make a short lane between safehouses that feels watched" . In response, the AI Copilot immediately drafts a `contractCard` based on the prompt's intent, presenting it to the designer for review . This initial proposal is not a final suggestion but a starting point. The card appears within the editor as an interactive UI element, typically featuring editable numeric inputs and sliders for each invariant and metric band, allowing the designer to fine-tune the parameters directly . For example, if the initial proposal for `DET` (Dread Event Threshold) seems too high, the designer can lower it, and the AI instantly recomputes all downstream suggestions—such as potential assets, PCG placement hints, and code snippets—to ensure they remain compliant with the newly adjusted contract . This tight feedback loop turns the editor into a powerful contract authoring tool, where the designer's instincts are translated into machine-verifiable targets.

A critical function of the AI Copilot is its mandatory awareness of the game's history and ledger . Before proposing any concrete entity, such as an NPC, item, or event, the assistant must execute a `historyQuery(regionId, radius)` API call to fetch relevant entries from the attested history layer . If this query reveals conflicting or overloaded history for the targeted area, the AI is obligated to reject naive actions and propose a contract-compliant alternative instead . This prevents the creation of narrative contradictions or gameplay-breaking content that might arise from ignoring the established lore and player journey. This behavior is a direct parallel to the rejection logic already implemented in the CI pipeline for signatures that violate invariant envelopes . Requests for random or uncontextualized content, such as "Spawn a monster here," are met with a structured refusal . Instead of complying, the AI returns a revised `contractCard` and a rationale explaining why the request was rejected and suggesting

a compliant alternative, such as creating an "almost-sighting" micro-seed tied to a valid `deadledgerref` that raises `SHCI` (Spectral-Harmonic Consistency Index) without violating safety caps or `DET` thresholds .

To support this workflow, the Unreal Editor plugin requires a suite of backend services. A `driftDetector(sceneSegment, contractCard)` service runs continuously in the background, computing live approximations of key metrics (e.g., scare counts, `DET` spikes, `ARR` changes) from the evolving scene and comparing them against the `contractCard`'s declared bands . When a value drifts out of bounds, the editor provides non-modal warnings directly in the UI, for example, through gutter badges or timeline markers, linking the warning to specific assets or events . These warnings include actionable fixes, such as "DET exceeded by 0.8 — reduce overt events; convert one event to an ‘almost-event’ chain," and may suggest an update to the contract card itself . This continuous validation ensures that the design remains aligned with its original intent throughout the iterative development process. The implementation checklist prioritizes this initial phase, focusing on establishing the contract card UI, integrating the schema validator, and providing the foundational ledger query API before moving on to more complex features . This phased approach allows for the incremental rollout of capabilities, ensuring each component is robust before building upon it.

Governance and Enforcement Across the Development Lifecycle

The unified contract framework is underpinned by a robust governance model that enforces doctrinal correctness from the moment a design idea is conceived in the editor until it is committed to the repository and executed by the game engine. This multi-layered enforcement mechanism leverages Horror.Place's existing infrastructure, including the Continuous Integration (CI) pipeline, the Authorization and Ledger Network (ALN), and the Dead-Ledger itself, to create a single, authoritative source of truth governed by a clear set of rules . The system's design prioritizes forward compatibility, mandating that all new workflows and tools adhere strictly to the new `contractCard` standard, while providing a gradual deprecation path for legacy systems to prevent long-term complexity . This ensures that the entire development lifecycle—from ideation to deployment—is aligned with the project's core architectural principles.

At the heart of this governance model is the `signature` document, which is used by the CI pipeline to validate commits . The `contractCard` is designed to be a literal subset of

this signature, embedded within it as a top-level field . This integration is governed by a strict "Subset Rule": the target values in the `contractCard.invariants` must lie strictly within the corresponding `invariantenvelope` ranges defined in the parent signature, and the `[min, max]` values in `contractCard.metrics` must exactly match those in the `metricenvelope` . During a pre-commit check, a utility like `schema_spine_scanner` performs a rigorous validation pass on all generated artifacts and the signature itself . If the `contractCard` fails to satisfy the subset rule, or if any generated artifact fails to validate against its schema, the commit is rejected . This makes the editor Copilot just another type of signature generator, whose output is subject to the same stringent checks as any other automated process . The `signature` document also carries `policyprofile` and `entitlementtier` information, which binds the `contractCard`'s numerical proposals to specific permissions and restrictions, preventing unauthorized actions like raising `DET` beyond a certain cap .

The ALN and Dead-Ledger provide the ultimate layer of authority, ensuring that only artifacts with proper provenance and attestation can be executed in the game world . The `deadledgerref` field in the `contractCard` serves as a pointer to a bundle or seed entry in the ledger, which has been verified and signed off according to the project's governance rules . Any operation that touches Tier-2 seeds or events is required to have a valid `deadledgerref` . The AI Copilot and all external agents must therefore query the ledger before generating any content that interacts with this tier; if no compatible history entry exists, they must refuse to act and suggest an appropriate alternative . This mirrors the entitlement model of the Dead-Ledger, where artifacts must be anchored to bundles and seeds that have proper ledger entries and policy classifications before activation . The entire system is built on a "No random spawns rule": any request that cannot be normalized into a `contractCard` with a valid history anchor is rejected by design, turning the AI into a guardian of narrative and gameplay coherence .

The governance framework extends to external agents and community-facing tools through a capability-based access control model [22] . Agents, whether internal AI assistants or external generators, can draft entirely new `contractCards`, but their freedom is constrained by ALN/Dead-Ledger capability tokens and routing profiles . These tokens define the outer bounds for allowable invariants, metric bands, and `styleids` for a given agent or user tier . An agent can only propose a `contractCard` whose parameters are a subset of the bounds granted by its capabilities. The proposed card is then submitted for validation, where the CI and ALN contracts either approve, adjust, or reject it based on policy and tier rules . This preserves creativity at the contract level while ensuring that the Schema Authority and Dead-Ledger governance remain firmly in charge of what can actually be executed anywhere in the VM-Constellation . This approach is analogous to Capability-Based Architecture, where operations are

authorized based on verifiable credentials rather than implicit trust, providing a secure and scalable model for distributed content generation [22] [37]. All of this is governed by a set of research-grade design rules that maintain alignment across the editor, CI, and Dead-Ledger layers, ensuring a cohesive and disciplined development environment .

Advanced Capabilities: Multi-Tiered Control and External Agent Integration

The proposed framework extends beyond managing individual seeds and scenes to encompass a sophisticated, multi-tiered control hierarchy that allows for both granular creative control and high-level strategic governance . This hierarchical model consists of three distinct layers—all expressed in the same invariant and metric language—which enables AI and designers to work at the appropriate level of abstraction. The system is also designed for extensibility, allowing external agents and even public users to participate in the content creation process within clearly defined, doctrinally sound boundaries . This capability transforms the framework from a simple editor tool into a comprehensive design and governance platform for the entire Horror.Place universe.

The three-tiered control structure provides a scalable mechanism for managing complexity and ensuring coherence across large-scale projects like regions and campaigns. At the highest level are **Global Policy Profiles**, which establish overarching rules and constraints for the entire project. These profiles define absolute caps on certain invariants (e.g., maximum `DET` spikes allowed in any part of the game) and dictate the available `styleids` and policy tiers . They represent the non-negotiable principles of the project's constitution, guiding all subsequent design decisions [1]. The second level is the **Regional or Campaign Envelope**. This is a higher-level contract that defines the contextual constraints and tonal expectations for a larger area of the game world . An AI assistant working on a new region would first read the regional envelope to understand the prevailing mood, historical context, and gameplay pacing before drafting dozens of individual seed-level cards. This ensures that the collection of seeds within a region forms a coherent whole. Finally, at the lowest level are the **Local Seed/Scene Cards**. These are the fine-grained specifications for individual game elements, as detailed in the core proposal, defining the precise invariants and metric bands for a specific scene or encounter . This hierarchy allows designers to focus on the micro-details of a scene while knowing their work is always bounded by the macro-context provided by global policies and regional envelopes.

The framework is architected to safely integrate external agents, which can range from specialized AI personas to third-party procedural generation tools . These agents are not given free rein; their capabilities are strictly controlled by ALN/Dead-Ledger capability tokens and routing profiles . When an agent submits a request, its token determines the outer bounds for the invariants and metric bands it can propose, as well as the `styleids` it can use . The agent can draft a new `contractCard` within these bounds, but the AI Copilot or a dedicated validation service must then check if the proposed card's target values are a subset of the envelope defined by the agent's capabilities. This ensures that while agents can propose novel and creative configurations, they cannot violate hard-coded policy constraints . This capability-based approach is crucial for maintaining security and doctrinal integrity in a distributed system, as it replaces implicit trust with verifiable credentials and explicit policies [76]. It allows the team to harness the power of external computation without compromising the quality and consistency of the final product.

Furthermore, because the `contractCard` is a purely numeric, schema-defined object, it can be safely exposed to a wider audience through public-facing tools without breaking the core doctrine . A web-based playground or a command-line interface (CLI) could allow community designers or external users to author `contractCards` by selecting values from predefined ranges and choosing from a library of `styleids` . Such a tool would run the same JSON Schema validations as the core CI pipeline, refusing any card that violates tier or policy constraints . Upon receiving a valid `contractCard`, the tool would return only schema-valid outputs: seed stubs with IDs and envelopes, and examples of procedural content generation (PCG) or metric hooks written in Lua or Blueprint syntax and tied directly to the card's identifiers . This approach democratizes content creation by abstracting away the raw horror content and focusing on the underlying structure and feel of a seed. The outputs are inherently safe because they contain only references, IDs, and calls to narrow APIs (`H.Metrics`, `H.PCG`), never raw trauma payloads, thus respecting the Rivers-of-Blood constraints and maintaining GitHub compliance [40] [51]. This creates a pathway for community contribution that strengthens, rather than weakens, the central governance model.

Concrete Outputs and Telemetry-Driven Iteration

The unified contract framework generates a suite of small, schema-aligned artifacts that are guaranteed to be compliant with the `contractCard` and ready for CI validation . These outputs serve as the direct translation of the designer's intent into executable game

logic and data structures. Beyond immediate generation, the system incorporates a powerful telemetry and learning loop that captures data from both the editor and playtests. This data is used to correlate contract configurations with player behavior, enabling the AI Copilot to evolve from a simple generator into a research-backed critic that can offer evidence-based recommendations and warn against historically problematic designs . This closed-loop feedback system is essential for the long-term improvement of the framework and the quality of the generated content.

When a designer accepts a `contractCard`, the AI Copilot generates several key artifacts: 1. **Registry Entry Stub:** A small JSON file is automatically created that registers the new seed or asset. This entry includes the stable `contractRef` that points back to the `contractCard`, along with the `styleid` and `schemaref` . This registry stub serves as the canonical entry point for the game engine to load and instantiate the new content. 2. **Procedural Content Generation (PCG) Rules:** The AI provides PCG helper functions, often in pseudocode or as small Lua snippets, that place assets and geometry according to the contract's targets . For instance, it might generate rules to place a "narrow bend" at an LSG hotspot or an "ambiguous landmark" near an AOS peak, using the `metricsTargets` from the contract card to guide the placement algorithm . 3. **Unreal Blueprint Fragments:** Small, focused Blueprint snippets are offered that demonstrate how to implement the desired logic. These fragments call canonical helper functions like `H.Metrics.Increment("encounter_count", schemaref="...")` or `H.Metrics.Adjust("SHCI", +0.05)`, ensuring that all metric tracking is centralized and contract-aware . The logic is gated by invariant checks, such as ensuring an event only occurs if `H.Metrics.DET < 4.0` . 4. **Lua Setup Scripts:** Lightweight Lua scripts are provided that treat the loaded `contract` as an object resolved via a narrow API like `H.Contract.load(contractRef)` . These scripts enforce ledger anchors by checking `H.History.isAnchored()` before allowing a seed to instantiate, acting as a runtime guardrail .

All of these generated artifacts are designed to be "CI-ready by construction." They embed the `schemaref`, `styleid`, and `deadledgerref` placeholders, which the project's CI pipeline can later validate against the registry and the Dead-Ledger protocol, ensuring every piece of content is properly documented and governed .

The true power of the framework lies in its ability to learn and adapt. The system establishes a telemetry pipeline that captures two streams of data. The first stream comes from the editor, logging every interaction: the designer's initial prompt, the AI-proposed `contractCard`, any edits made by the designer, and the final `contractCard` that is shipped . This creates a supervised dataset mapping natural language intent to numerical outcomes, which is invaluable for training the AI . The second stream comes from

playtests, where player behavior (backtracking, hesitation, time spent on evidence) and outcome metrics (UEC, ARR, CDL) are correlated with the `contractCard`s that produced them . Over time, the AI can use this aggregated data to surface evidence-backed recommendations, such as "Seeds with EMD bands of 0.6–0.8 and ARR > 0.8 have shown better player retention in Aral-basin tests" . The system also maintains a failure case database, flagging seeds that historically resulted in low engagement or high burnout (e.g., low UEC, high CDL) . The AI can then proactively warn designers when a proposed contract drifts into these known problematic zones, acting as a preventative critic. This continuous iteration loop, driven by ground-truth data, allows the AI to become a more effective and reliable co-designer over time, ultimately improving the quality and consistency of the horror experiences created [34] .

Implementation Roadmap and Strategic Synthesis

The successful implementation of the unified, schema-aware contract framework for Horror.Place requires a phased, strategic approach that prioritizes foundational stability and doctrinal purity. The roadmap begins with establishing the core schema and editor integration, progresses to enforcing governance through the CI pipeline, and culminates in leveraging telemetry for continuous improvement. This phased execution ensures that each component is built and validated before dependencies are introduced, mitigating risk and creating a robust system that can support both internal development and future external integrations. The synthesis of these efforts is a transformative shift in the development paradigm, elevating the AI from a content-generating tool to a disciplined, doctrine-enforcing partner deeply embedded in the project's core architecture.

The implementation can be broken down into a prioritized series of phases, closely aligned with the initial checklist provided . **Phase 1** focuses on laying the groundwork: writing the `coreschemascontractcardv1.json` schema, extending the main `coreschemassignaturev1.json` to embed the contract card, and wiring the schema into the validation pipeline. Concurrently, a prototype of the Unreal Editor Copilot is developed, centered on the UI for the `contractCard` and the core logic to translate designer prompts into an initial draft of the card, complete with a validator . This phase is critical for establishing the shared language and ensuring that all subsequent components can interoperate seamlessly. **Phase 2** builds on this foundation by enabling the full contract-first workflow. The AI is enhanced to generate registry stubs, PCG rules, and small code fragment templates (Blueprint pseudocode, Lua) once a contract is accepted, with all artifacts correctly referencing `schemaref`, `styleid`, and

`deadledgerref` . **Phase 3** introduces the dynamic aspects of the framework: a background `driftDetector` service that provides real-time warnings in the editor, and a pre-commit `schema_spine_scanner` that enforces contract compliance before code can be saved or pushed .

Phase 4 shifts focus to the learning and research loop. A telemetry pipeline is instrumented to log editor interactions (prompts, proposals, edits) and playtest outcomes (telemetry data, designer notes), creating the ground-truth dataset needed for analysis . This data is used to train the AI to better match the preferences of expert designers and to surface evidence-backed recommendations, transforming the assistant from a generator into a critic . Finally, **Phase 5** focuses on hardening the system's security and governance. Content guardrails are rigorously tested, and the CI policy enforcement is tightened to ensure all generated artifacts fully comply with the schema authority and Dead-Ledger protocols . Throughout this process, a clear migration strategy for legacy signatures must be developed, involving audit reports that flag deprecated formats and documentation to guide designers through the transition, ensuring a smooth move toward the single authoritative `signature with embedded contractCard` model .

In synthesis, this framework represents a significant architectural and cultural evolution for Horror.Place. By codifying design intent into a `contractCard` validated by a shared JSON Schema, the project achieves unprecedented levels of consistency, quality, and safety. The AI Copilot becomes a true co-designer, speaking the language of the doctrine and enforcing invariants at every step. The multi-tiered governance model, combining global policies, regional envelopes, and local contracts, provides a scalable way to manage complexity while preserving creative freedom. The integration with the CI pipeline and the Dead-Ledger ensures that doctrinal correctness is not just a guideline but a hard requirement, auditable and enforceable. Ultimately, this system moves the development process away from subjective "vibes" and toward objective, data-driven creation. It empowers designers by giving them a powerful, interactive tool for authoring constraints, protects the project from inconsistent or low-quality content, and provides a clear path for AI to contribute meaningfully to the art of horror game design.

Reference

1. Diving Into Spec-Driven Development With GitHub Spec Kit https://developer.microsoft.com/blog/spec-driven-development-spec-kit

2. Documentation - Apache Unomi 2.x https://unomi.apache.org/manual/latest/index.html
3. UniSAFE: A Comprehensive Benchmark for Safety Evaluation of ... https://arxiv.org/html/2603.17476v1
4. [PDF] EvolveSearch: An Iterative Self-Evolving Search Agent https://faculty.sist.shanghaitech.edu.cn/faculty/tukw/emnlp25-search.pdf
5. [PDF] IWSDS 2026 The 16th International Workshop on Spoken Dialogue ... https://aclanthology.org/2026.iwsds-1.pdf
6. An Open-source Testbed for Generative Challenges in Games https://dl.acm.org/doi/10.1145/3723498.3723794
7. [PDF] A modular framework for automated evaluation of procedural ... - arXiv https://arxiv.org/pdf/2505.16801
8. Creating a Newer and Improved Procedural Content Generation ... https://www.mdpi.com/2073-431X/13/11/304
9. Procedural Content Generation via Machine Learning (PCGML) https://ieeexplore.ieee.org/document/8382283/
10. Search-based procedural content generation for GVG-LG https://www.sciencedirect.com/science/article/abs/pii/S1568494619306908
11. Procedural Content Generation in Games | Springer Nature Link https://link.springer.com/book/10.1007/978-3-319-42716-4
12. Management, Innovation and Entrepreneurship in Challenging ... https://link.springer.com/content/pdf/10.1007/978-3-031-47164-3.pdf
13. [PDF] Advance and Innovative Research - ResearchGate https://www.researchgate.net/profile/Firdaus-Khan-2/publication/338101627_IS_MARKET_CAPITALIZATION_BASED_INVESTING_A_GOOD_STRATEGY_-_A_MULTIPLE_REGRESSION_ANALYSIS_OF_RANDOMLY_SELECTED_BSE_STOCKS/links/5dfe3a08299bf10bc36ead1c/IS-MARKET-CAPITALIZATION-BASED-INVESTING-A-GOOD-STRATEGY-A-MULTIPLE-REGRESSION-ANALYSIS-OF-RANDOMLY-SELECTED-BSE-STOCKS.pdf
14. [PDF] HCI in Games - ResearchGate https://www.researchgate.net/profile/Olivier-Gavarry/publication/372229826_Escapad_Dance_Serious_Game_Designing_a_Therapeutic_Tool_for_Elderly_with_Mild_Cognitive_Impairment/links/64ec9fb340289f7a0fbffb0a/Escapad-Dance-Serious-Game-Designing-a-Therapeutic-Tool-for-Elderly-with-Mild-Cognitive-Impairment.pdf

15. Figma's AI integration for design and collaboration - LinkedIn https://www.linkedin.com/posts/davidkossnick_how-figma-integrates-ai-to-transform-design-activity-7357131716945145856-LJ5z
16. I'm Building My Own Coding Agent Harness (And It's Pretty Cool) https://dev.to/composiodev/im-building-my-own-coding-agent-harness-and-its-pretty-cool-1lpf
17. Trust in AI: progress, challenges, and future directions - Nature https://www.nature.com/articles/s41599-024-04044-8
18. [PDF] Enforcing Security by Construction in AI-Assisted Code Generation https://www.arxiv.org/pdf/2602.02584
19. Context-Aware Smart Contract Comment Generation Using ... https://www.sciencedirect.com/science/article/abs/pii/S0957417426009619
20. SolGen: Secure Smart Contract Code Generation Using Large ... https://dl.acm.org/doi/10.1145/3717383.3717394
21. A generative AI cybersecurity risks mitigation model for code ... https://www.nature.com/articles/s41598-025-34350-3
22. Capability-Based Architecture: A Practical Guide to Portability ... https://dev.to/gd-tech-guru/capability-based-architecture-a-practical-guide-to-portability-isolation-and-ai-readiness-2g4h
23. Generative AI-Driven Smart Contract Optimization for Secure ... - MDPI https://www.mdpi.com/2624-6511/8/4/118
24. [PDF] Ledger-Verified Run-Wise Certificates under Local DP - arXiv https://www.arxiv.org/pdf/2509.10550
25. Verifiable Credentials Data Model v2.0 - W3C https://www.w3.org/TR/vc-data-model-2.0/
26. A Hybrid Multi-Agent System for Early Scam Detection in Crypto ... https://www.mdpi.com/2076-3417/16/7/3122
27. Decentralized Oil Extraction Infrastructure via Blockchain-Enabled ... https://www.sciencedirect.com/science/article/pii/S2096720925001526
28. ORIGINAL UNEDITED MANUSCRIPT - Oxford Academic https://academic.oup.com/nsr/advance-article-pdf/doi/10.1093/nsr/nwag108/66901829/nwag108.pdf
29. ACloudGuru-博客中文翻译-四- - 绝不原创的飞龙 https://www.cnblogs.com/apachecn/p/18518215
30. Matt Barrett on Building Entrepreneurial Culture in Capital Markets https://www.linkedin.com/posts/adaptive-ltd_entrepreneurship-leadership-capitalmarkets-activity-7404842617454153729-uemH

31. Visual Studio Code introduces Auto Mode for AI models - LinkedIn https://www.linkedin.com/posts/burkeholland_introducing-auto-mode-in-visual-studio-code-activity-7376726346854461440-X18I
32. GitHub Copilot Extension Chat function not appearing in VSCode https://stackoverflow.com/questions/75922278/github-copilot-extension-chat-function-not-appearing-in-vscode
33. Learn APIs for AI Agents in a No-Code Environment - LinkedIn https://www.linkedin.com/posts/nateherkelman_apis-for-ai-agents-the-only-beginners-guide-activity-7326978260330860545-fbE8
34. [PDF] Building Cognitive Applications with IBM Watson Services: Volume 1 ... https://www.redbooks.ibm.com/redbooks/pdfs/sg248387.pdf
35. Procedural Content Generation in Games: A Survey with Insights on ... https://arxiv.org/html/2410.15644v1
36. (PDF) Base—Units—Tables—Invariants: Sumerian Computation ... https://www.researchgate.net/publication/395454127_Base-Units-Tables-Invariants_Sumerian_Computation_and_Its_Modern_Echoes
37. TessPay: Verify-then-Pay Infrastructure for Trusted Agentic Commerce https://arxiv.org/html/2602.00213v1
38. Cybersecurity Curriculum Overview | PDF | Engineering - Scribd https://www.scribd.com/document/842688013/B-E-cse-Cyber-Security-Padeepz-App
39. BACE: LLM-based Code Generation through Bayesian Anchored ... https://arxiv.org/html/2603.28653v1
40. LLM-based JSON mapping and blockchain integration for Digital ... https://www.sciencedirect.com/science/article/pii/S2666954426000050
41. LLM-based and Retrieval-Augmented Control Code Generation https://dl.acm.org/doi/10.1145/3643795.3648384
42. [PDF] A2ASECBENCH: A PROTOCOL-AWARE SECURITY BENCHMARK ... https://openreview.net/pdf?id=LfdFnakqGJ
43. The 2025 Conference on Empirical Methods in Natural Language ... https://aclanthology.org/events/emnlp-2025/
44. Secure and Intelligent Low-Altitude Infrastructures: Synergistic ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12609838/
45. Trustworthy Data Space Collaborative Trust Mechanism Driven by ... https://www.mdpi.com/2078-2489/16/12/1066
46. High Dimensional Procedural Content Generation - arXiv https://arxiv.org/html/2602.18943v1

47. (PDF) Procedural Content Generation for Game 3D Modeling https://www.researchgate.net/publication/396808227_Procedural_Content_Generation_for_Game_3D_Modeling_The_Investigation_of_AI-Based_Approaches_for_Improving_Game_Experience
48. Constraint Is All You Need: Optimization-Based 3D Level ... https://dl.acm.org/doi/10.1145/3723498.3723840
49. SpecEval: Evaluating Model Adherence to Behavior Specifications https://arxiv.org/html/2509.02464v1
50. Entertainment Computing and Serious Games - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-030-34644-7.pdf
51. Schema-Validated, Rules-Guided, and Policy-Driven ... https://apievangelist.com/2025/10/28/schema-validated-rules-guided-and-policy-driven-experiences/
52. Explanation of $dynamicRef$ dynamicAnchor in JSON ... https://stackoverflow.com/questions/69728686/explanation-of-dynamicref-dynamicanchor-in-json-schema-as-opposed-to-ref-and
53. 10 AI-Powered VSCode Extensions That Changed My Workflow ... https://dev.to/ilsa_shaikh_089e2bfab0bf4/10-ai-powered-vscode-extensions-that-changed-my-workflow-4959
54. (PDF) SecRepoBench: Benchmarking LLMs for Secure Code ... https://www.researchgate.net/publication/391328959_SecRepoBench_Benchmarking_LLMs_for_Secure_Code_Generation_in_Real-World_Repositories
55. A Review of TRiSM Frameworks in Artificial Intelligence Systems ... https://onlinelibrary.wiley.com/doi/10.1111/exsy.70213
56. Cryptographic Binding and Reproducibility Verification for AI Agent ... https://arxiv.org/html/2603.14332v1
57. 虚幻引擎5.0版本说明 https://dev.epicgames.com/documentation/zh-cn/unreal-engine/unreal-engine-5.0-release-notes?application_version=5.0&ciYWxj=hq54y5
58. A Survey of Reinforcement Learning For Economics - arXiv https://arxiv.org/html/2603.08956v5
59. A hierarchical deep learning and distributionally robust framework ... https://www.sciencedirect.com/science/article/pii/S2590123025045773
60. Arxiv今日论文| 2026-02-16 - 闲记算法 http://lonepatient.top/2026/02/16/arxiv_papers_2026-02-16
61. AI vs Human Cost Efficiency in Call Centers | PDF - Scribd https://www.scribd.com/document/903682546/ChatGPT-AI-vs-Human-Cost-LLM-Orchestrator
62. Blogs March 2026 - ACM Queue https://queue.acm.org/blogs.cfm?archdate&theblog=24

63. words_SG_upto2020.txt - Zenodo https://zenodo.org/record/5516252/files/words_SG_upto2020.txt
64. Dicionario portugues - Academia.edu https://www.academia.edu/32592435/Dicionario_portugues
65. BlockA2A: Towards Secure and Verifiable Agent-to-Agent ... - arXiv https://arxiv.org/html/2508.01332v3
66. AIRPoC: An AI-Enhanced Blockchain Consensus Framework ... - MDPI https://www.mdpi.com/2079-9292/14/20/4058
67. A Zero-Knowledge Proof-Enabled Blockchain-Based Academic ... https://www.researchgate.net/publication/392256915_A_Zero-Knowledge_Proof-Enabled_Blockchain-Based_Academic_Record_Verification_System
68. Chaos Engineering of Ethereum Blockchain Clients https://dl.acm.org/doi/10.1145/3611649
69. [PDF] Intelligent World 2030 - Huawei https://www-file.huawei.com/-/media/corp2020/pdf/giv/2024/intelligent_world_2030_2024_en.pdf?la=en
70. [PDF] Cryptographic Binding and Reproducibility Verification for AI Agent ... https://arxiv.org/pdf/2603.14332
71. [PDF] A semantic framework for defining and assessing e-identity ... https://theses.hal.science/tel-05554605v1/file/2025TLSES209.pdf
72. Json Schema not evaluating if else condition properly in ... https://stackoverflow.com/questions/78610635/json-schema-not-evaluating-if-else-condition-properly-in-draft-version-2020-12
73. A Case Study on JSON Schema https://openreview.net/forum?id=p84kZ3ZFux&referrer=%5Bthe%20profile%20of%20Taylor%20Berg-Kirkpatrick%5D(%2Fprofile%3Fid%3D~Taylor_Berg-Kirkpatrick1)
74. [PDF] the eu video games software industry - JRC Publications Repository https://publications.jrc.ec.europa.eu/repository/bitstream/JRC60711/reqno_jrc60711_jrc60711.pdf.pdf
75. From prompt injections to protocol exploits: Threats in LLM-powered ... https://www.sciencedirect.com/science/article/pii/S2405959525001997
76. [PDF] Agentic AI Identity and Access Management - OpenReview https://openreview.net/pdf/7db8d7d31396bd9a8cc21dbbc479c7511639f8d8.pdf
77. Arxiv今日论文| 2026-02-12 - 闲记算法 http://lonepatient.top/2026/02/12/arxiv_papers_2026-02-12
78. Simple Index https://mirrors.aliyun.com/pypi/simple/
79. A Deep Dive into AI Ethics and Regulations - arXiv https://arxiv.org/html/2412.04683v1

80. cmnt_vocab.txt - CMU School of Computer Science https://www.cs.cmu.edu/~ark/blog-data/data/blog_data_v1_0/dk/hbc_data/data/cmnt_vocab.txt

81. Azure updates | Microsoft Azure https://azure.microsoft.com/updates?id=526665?WT.mc_id=AZ-MVP-5004796